Hello World in Three Environments

15 points / due October 28, 2026 at 12:35 PM Pacific



Stage 3 / Independent study

Investigate the lecture and each build. Use documentation, your team and AI; test your ideas and ask focused instructor questions.

ONE MESSAGE / THREE BUILDS

Console	WPF / XAML	Game engine
Print to console	Click and display	Object + script

The shared class returns a string. Each environment chooses the output.

Investigate every step

For every step: predict -> try -> observe -> explain. Keep a short note and an evidence file/link, including during the lecture. If something fails, keep the error and your next step.

Finish: Console source + WPF source + engine attempt + one investigation README + evidence + receipt, committed and pushed to GitHub. Submit the repository URL to Canvas.

Open the five-step workbook

Lecture / investigate before building

Keep three concepts, one prediction, one question and lecture evidence. A whiteboard/slide photo or annotated sketch is suitable evidence.

0-2 min / investigate the lecture

Write three concepts, predict one output and record one question. Keep a whiteboard/slide photo or annotated sketch as evidence. The lecture is Investigation 0.

2-4 min / type and test

Type short blocks, predict, run and read the result. Notice braces, parentheses and semicolons. Assistive input and recovery copying remain available; explain what the code does.

4-7 min / class, object and method

Greeting describes a type. new Greeting creates an object. Name stores data. BuildMessage is an instance method that returns a string. A variable of this class type holds an object reference.

7-10 min / same logic, different output

Console.WriteLine writes to the console. XAML describes WPF controls; a C# Click handler sets MessageText.Text. An engine script uses the engine's output/log or screen API.

10-13 min / events and lifecycle

A button calls a handler through its Click event. Unity calls Start; Godot calls _Ready. The base class and callback belong to the chosen engine. x:Name identifies a WPF control; it is not a data binding.

13-15 min / release to investigation

Whiteboard class -> object -> method -> output. Work solo or choose a group with a team lead and task list. Attempt all three builds; keep notes and evidence as you go.

Whiteboard / Attention-Driven Design

Draw class -> object -> method -> output. Identify the user and need. Visual hat: what is shown. Logical hat: the method/event. Data hat: the stored name/message. Photograph your work and record one concise Discord peer check.

Solo or group

A group must name a team lead and keep a task list with Task | Owner | Status. The lead coordinates Git/access and progress. List members and their contributions in README. Each learner submits the link and explains their own work.

00 / PRE-START

Open the course GitHub repository; confirm instructor access. Open the IDE and check edit/run/save/commit/push. Keep Console, WPF, EngineTrial and Evidence in separate folders on your approved data drive. Use your own character/user or the VOID RUNNERS beacon context.

Investigation 1 / Console Hello World

1. Create a C# Console App named HelloConsole in Visual Studio. Select an installed supported .NET target and your data-drive folder.

2. Add a class file Greeting.cs to the project. Type the class below. Replace Program.cs with the caller below.

3. Predict the output, run with Ctrl+F5 and capture the console. Change Name and rerun. Keep your note and screenshot.

Greeting.cs

```
namespace HelloInvestigation
{
    public class Greeting
    {
        public string Name { get; set; } = "runner";

        public string BuildMessage()
        {
            return $"Hello, world, {Name}!";
        }
    }
}
```

Program.cs

```
using HelloInvestigation;

Greeting greeting = new Greeting();
greeting.Name = "Ari";
Console.WriteLine(greeting.BuildMessage());
```

Expected: Hello, world, Ari!

class: type description. **new:** create an object. **Name:** data exposed by a property. **BuildMessage:** method returning text. **Console.WriteLine:** display the returned string. Braces show which members belong inside the class.

Official Console guide

Investigation 2 / XAML window anatomy

1. Create a separate C# WPF Application named exactly HelloWpf for this example. Use the .NET desktop development workload and an installed supported .NET target.

2. Add your Greeting.cs to this WPF project. Keep its HelloInvestigation namespace.

3. Open MainWindow.xaml. Type the Window shell and empty StackPanel first; keep both XML namespaces. Add the TextBlocks, then the Button. Compare with the complete XAML below.

```
<Window x:Class="HelloWpf.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Hello World Investigation" Width="520" Height="280"
    Background="#0A202B">
    <StackPanel Margin="28">
        <TextBlock Text="One greeting. Three environments."
                   Foreground="#EFD09B" FontSize="18" />
        <TextBlock x:Name="MessageText" Text="Ready to investigate."
                   Foreground="White" FontSize="24"
                   TextWrapping="Wrap" Margin="0,20,0,20" />
        <Button x:Name="HelloButton" Content="Say Hello"
                Click="SayHello_Click" Padding="12,8"
                HorizontalAlignment="Left" />
    </StackPanel>
</Window>
```

BUTTON TO OUTPUT

XAML	Click event	C# output
Button + MessageText	SayHello_Click	MessageText.Text

x:Name identifies a control. Click names the handler; it is not data binding.

Window is the frame. StackPanel arranges controls. TextBlock displays text. Button begins an action. The next page connects the visual controls to C#.

Official WPF Hello World guide

Investigation 2 / C# behind the window

Expand MainWindow.xaml in Solution Explorer and open MainWindow.xaml.cs. Keep the constructor and InitializeComponent; add SayHello_Click inside the MainWindow class. The complete paired file follows.

```
using System.Windows;
using HelloInvestigation;

namespace HelloWpf
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void SayHello_Click(object sender, RoutedEventArgs e)
        {
            Greeting greeting = new Greeting();
            greeting.Name = "Ari";
            MessageText.Text = greeting.BuildMessage();
        }
    }
}
```

Match: x:Class in XAML = HelloWpf.MainWindow in C#. partial joins generated XAML code with this file. InitializeComponent creates the controls. The handler calls your Greeting method and assigns the result to MessageText.Text.

Run with F5. Click Say Hello. Capture the running window showing the greeting. Explain what XAML describes and what the C# handler changes.



Reference: the actual lesson XAML rendered after its Click event was tested in-process.

Investigation 3 / Unity object and script

Use this route if Unity is your selected engine. Otherwise choose the Godot route or the equivalent official guide for your engine. Record edition/version, language, setup and scripting links, SDK/editor, project path and Git ignore plan.

1. Open an approved blank 3D Unity project. Add GameObject -> 3D Object -> Cube. Name it Beacon; set position 0,0,0 and keep a camera looking at it. Save the scene.

2. Create Assets/Scripts. Add your Greeting.cs. Do not copy Program.cs or the console project file.

3. Create HelloWorld.cs as a MonoBehaviour script. Type the code below. Attach it to Beacon; confirm it appears in Inspector.

4. Open Window -> General -> Console. Enter Play mode. Capture the scene object, attached script and greeting in Console, or the exact error and next step.

```
using UnityEngine;
using HelloInvestigation;

public class HelloWorld : MonoBehaviour
{
    private void Start()
    {
        Greeting greeting = new Greeting();
        greeting.Name = "Ari";
        Debug.Log(greeting.BuildMessage());
    }
}
```

Unity calls Start on the attached active component. MonoBehaviour is the base class; Greeting is your plain class. Debug.Log writes to the Unity Console. A changed editor property alone is not the required Hello World script attempt.

Official Unity script/attachment guide

Keep: scene and script source, object/attachment evidence, output or error, investigation notes. A documented attempt is required; do not claim a successful run you did not observe.

Investigation 3 / Godot or another engine

1. Use Godot .NET plus its required SDK. Create a 2D Scene (Node2D), add a Sprite2D child named Beacon and save the scene.

2. Give Beacon a texture: drag the project icon.svg into Texture, or use an approved image. Set Position to 160,120.

3. Add Greeting.cs in a Scripts folder. Attach HelloWorld.cs to Beacon using the Sprite2D base type; type the code below.

4. Build and run the current scene. Capture the object, attached script and greeting in Output, or the exact error and next step.

```
using Godot;
using HelloInvestigation;

public partial class HelloWorld : Sprite2D
{
    public override void _Ready()
    {
        Greeting greeting = new Greeting();
        greeting.Name = "Ari";
        GD.Print(greeting.BuildMessage());
    }
}
```

Sprite2D is the base type; partial supports generated engine code. Godot calls _Ready; GD.Print writes to Output. This file is not a Unity script.

Godot C# guide

Another engine / same attempt

Follow its official scene and scripting guide. Place an object/entity/instance. Add/attach a Hello World script or native scripted event. Attempt to run and capture the message on screen or in debug/output. Name the actual language and command. All learners still complete the C# Console and WPF investigations.

Official engine comparison links

Investigation record / notes and evidence

USE THIS FOR EVERY STEP

Predict	Try + observe	Explain
What should happen?	What actually happened?	Why / next step?

Include the lecture. Keep a screenshot, photo, annotated sketch or evidence file/link.

One README / four entries

Lecture: three concepts, a prediction, a question and evidence.
Console: prediction, code typed, output, explanation and screenshot.
XAML: UI/code connection, click result, explanation and screenshot.
Engine: setup sources, object/script, run attempt, output/error and next step.

Required AI / investigate one answer

I am investigating Hello World in C# Console, WPF/XAML and [engine/version]. My class is Greeting and BuildMessage returns a string. I expected [prediction] but observed [result or exact error]. Explain one relevant class/object/event/callback connection and one small next test. Give an official documentation link. Do not build the whole assignment for me. I will record the source check, test and what I keep/change/reject.

No live AI? Verify this supplied response

Supplied response to verify: Greeting describes a type; new Greeting creates an object. BuildMessage returns text to its caller. Console.WriteLine displays it in a console. In WPF a Click handler can assign it to MessageText.Text. In Unity an attached MonoBehaviour can log it from Start; in Godot an attached node script can print it from _Ready. Check that the object is in the running scene and that the script is attached. Verify the chosen engine's callback and output API in its official documentation.

Keep one useful answer point, your official-source check, a test and the keep/change/reject decision. No paid account or private chat export is required.

If you are blocked

Keep the actual code, error and attempted fix. Ask a focused question and record a next step. Request an approved alternative for access/setup barriers. A failed run can still provide evidence; do not mark an unobserved result successful.

Submit / one clear GitHub package

R1 / Investigate the lecture and plan / 2 points

Keep lecture notes: three concepts, one prediction and one question. Photograph your class whiteboard (class -> object -> method -> output) and record a concise Discord peer check. If working as a group, name a team lead and keep a task list with owners/status; identify your own contribution.

R2 / Console Hello World / 3 points

Create a C# Console App. Type the Greeting class and Program.cs; instantiate the class and print Hello, world. Keep the editable project, output screenshot and a brief prediction/result/explanation note.

R3 / WPF / XAML Hello World / 3 points

Create a separate C# WPF Application. Build a small XAML window with a text area and button; connect its Click handler to C# and display Hello, world. Keep the project, running-window screenshot and a short note explaining XAML versus code-behind.

R4 / Game Engine Hello World attempt / 4 points

Research and set up your chosen engine. Place an object in a scene, attach/add a Hello World script or native scripted event, then attempt to run. Show Hello, world on screen or in the debug/output log. Keep source, the scene/object/script evidence, output or exact error, official links and your next step. Verify one focused AI answer or the supplied response. A documented attempt is required; label a blocked run honestly.

R5 / GitHub package and receipt / 3 points

Commit and push Console, WPF and EngineTrial source, one investigation README, Evidence and the downloaded 10B receipt to GitHub. Include team lead/task list only if working as a group. Check instructor access and submit the GitHub repository URL to this Canvas Action. Reuse class evidence; no duplicate report.

Before the deadline

1. Download your investigation notes as README. Keep Console/, WPF/, EngineTrial/ and Evidence/. Group lead/task list belongs in README.
2. Download cs077-10b-receipt.txt into Evidence/. Commit and push. Skip generated caches, bin/obj/builds and private account data.
3. Check instructor access, submit the GitHub repository URL to Canvas and verify confirmation.

Return to the Canvas Action

15 counted points; same deadline. 10A + 10B = 25 with separate Action submissions. Receipt is a learner declaration, not grade/attendance verification or a Canvas submission. Reuse class evidence; no duplicate report.